DAY 26 | HANDS-ON ACTIVITIES

Agentic AI and Agentic RAG: Problem Statements

Seven problems. All run offline with stand-ins; with an API key, the Agents SDK and model-based grading run for real.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Workflow or Agent?

Level: Warm-up | **Concepts:** design judgement

For each task, choose workflow or agent and justify: invoice data extraction; investigating why a KPI dropped; weekly report generation; customer onboarding checklist; researching a competitor.

Sample run / expected result

```
Invoice extraction: workflow (fixed steps)
KPI investigation: agent (path depends on findings)
```

Hints

- Ask: can I draw the steps before starting?

Problem 2: Agent from Scratch

Level: Easy | **Concepts:** agent loop, components

Use the notebook's MiniAgent with two tools. Add a memory list that records every tool call and a stop condition after 5 steps.

Sample run / expected result

```
step 1 get_order_status -> delivered
step 2 calculate_refund -> 720
memory: 2 entries
final: refund 720
```

Hints

- Keep instructions, tools and memory as separate attributes.

Problem 3: Agents SDK with a Handoff

Level: Medium | **Concepts:** Agent, function_tool, handoffs

With an API key, build a triage agent with an order tool and a handoff to a refunds agent. Run three questions and read the trace.

Sample run / expected result

```
"Where is ORD-2077?" -> triage answers
"I want a refund for ORD-1042" -> handed to Refunds
```

Hints

- Use await Runner.run(...) in notebooks.

Problem 4: Corrective RAG

Level: Medium | **Concepts:** grading, query rewrite, fallback

Run the CRAG graph on five questions, including one that needs a query rewrite and one that needs the external fallback. Print each path.

Sample run / expected result

```
"How much leave can I roll over?" -> retrieve -> grade: 0 relevant -> rewrite -> retrieve ->
```

Hints

- Print the graded documents to see why they passed or failed.

Problem 5: Better Grading

Level: Challenging | **Concepts:** LLM grader, thresholds

Replace the offline overlap grader with an LLM yes/no grader (online) and compare decisions on 10 question-document pairs you label yourself.

Sample run / expected result

```
Overlap grader agrees with labels: 7/10
LLM grader agrees: 9/10
```

Hints

- Label first, then measure.

Problem 6: Supervisor and Specialists

Level: Challenging | **Concepts:** multi-agent LangGraph

Extend the supervisor graph with a fourth specialist, a "checker" that verifies numbers in the writer's draft against the analyst's results.

Sample run / expected result

```
path: supervisor -> researcher -> analyst -> writer -> checker -> done
```

Hints

- The supervisor decides from what is missing in the state.

Problem 7: Agent Guardrails

Level: Stretch | **Concepts:** limits, approvals

Add three guardrails to MiniAgent: maximum steps, a blocklist of tools needing approval, and an output check that blocks answers containing an internal email domain.

Sample run / expected result

```
Blocked: tool issue_refund requires approval
Blocked: output contained internal email
```

Hints

- Fail closed: when unsure, block and explain.

Ganbatte (頑張って): do your best